🗒️ Assignment Brief: Mentoring Call Scheduling System

You will be building a mentoring call scheduling platform with RBAC (Role-Based Access Control) for Users, Mentors, and Admins. The goal is to create an experience that is highly intuitive, similar to Cal.com & acuityscheduling.com

🎯 Objective & Evaluation Criteria

You will be given an existing codebase with multiple dependencies, configurations, and middleware.

This assignment evaluates:

- Your ability to understand an existing repository quickly
- Your skill in modifying systems without breaking compatibility
- Your ability to think from a product perspective
- Your speed in shipping features within constraints

📌 What You Need to Do

1. Understand the Current Flow

- Here is a video explaining:
    - Current flow
    - Expected flow

https://www.tella.tv/video/assignment-brief-mentoring-call-scheduling-system-mentorque-cg2o

---

2. Refactor & Implement RBAC

- You may:
    - Clean up or replace existing middleware
    - Implement your own RBAC system with separate login pages. No need to sign up. Fill the data with a script initially. 10 users - 5 mentors - 1 admin
        - Users
        - Mentors
        - Admins
- Database:
    - Use Supabase or NeonDB
    - You are free to design:
        - Schema
        - Models & DDL

3. Core Product Flow (FINAL)

👤 User

- Adds:
    - Availability

❌ Cannot book calls

👩‍🏫 Mentor

- Adds:
    - Availability only
- ❌ Cannot book calls

🛠️ Admin

- Full control over system
- Responsible for:
    - Managing mentor metadata (tags + descriptions)
    - Viewing user requirements
    - Viewing mentor recommendations
    - Checking availability overlap
    - Booking calls

---

3 types of calls:

- **Resume Revamp:** Ideally, we would want a mentor who is from big tech.
- **Job Market Guidance:** A mentor who is good at communication.
- **Mock Interviews:** Someone from the same domain as the user.

🤖 4. RAG-Based Recommendation System

Implement a basic RAG system. *RAG implementation is not compulsory as long as the recommendation works fine with normal AI model API calls.*

- Use:
  - Free embedding models from Hugging Face
  - pgvector on NeonDB *(or vector-less approach)*
- Reference:
  - https://github.com/piyush-hack/pageindex-ts

🧠 Matching Logic

Mentor Data (Managed by Admin)

- Tags:
  - Tech / Non-tech
  - Big company / Public company
  - India / Ireland
  - Senior Developer
  - Good communication
- Description:
  - Added/controlled by Admin

User Data

- Tags:
  - Tech / Non-tech
  - Good communication
  - Asks a lot of questions
- Description:
  - Added by user

🎯 Expected Behavior

- When a user is ready for scheduling:
    - Admin sees:
        - Recommended mentors
        - Based on:
            - Tags
            - Descriptions
            - Vectorless RAG
- System should:
    - Check availability overlap
- Admin:
    - Selects mentor
    - Books the call

🧠 Key Focus Areas

- Product thinking
- Working with existing codebases
- Clean architecture
- Fast execution

https://github.com/mentorque/availabilitytrackerfrontend
https://github.com/mentorque/availability-trackerbackend

- Clone the above repositories
- You are **completely free** to:
    - Refactor the codebase
    - Modify architecture
    - Change the flows completely

👉 The goal is to make the system:

- **Functional**
- **Intuitive**
- While maintaining **backend compatibility wherever possible**

📝 Notes on Existing Repository

- For **user login**, the current implementation is access based on OAuth.
  👉 **Remove this completely** and keep authentication **simple (basic JWT login for now)**.
- The database should be **pre-filled with dummy data**:
    - **5 mentors** with:
        - Descriptions
        - Tags
    - **10 users** with:
        - Tags
        - Descriptions
    - And 1 Admin

- There are environment dependencies such as:
    - `VITE_SUPABASE_ANON_KEY`
    - `JWT_SECRET` used in middleware
- 👉 **Completely remove these dependencies** and simplify the middleware accordingly.

🗒️ Backend `.env` (Example)

```
# Database
DATABASE_URL=your_database_connection_string

# JWT
JWT_SECRET=your_jwt_secret
JWT_EXPIRES_IN=30d
MAIN_SITE_JWT_SECRET=your_other_secret_if_needed

# Server
PORT=5000
NODE_ENV=development

# OAuth (example)
GOOGLE_CLIENT_ID=your_google_client_id
GOOGLE_CLIENT_SECRET=your_google_client_secret
GOOGLE_REDIRECT_URI=http://localhost:5000/api/auth/google/callback
GOOGLE_REFRESH_TOKEN=your_refresh_token

# Frontend
FRONTEND_URL=http://localhost:5173

# Admin creds
ADMIN_EMAIL=admin@example.com
ADMIN_PASSWORD=admin123
ADMIN_NAME=Admin User
```

🗒️ Frontend `.env` (Example)

```
VITE_API_URL=http://localhost:5000
VITE_SUPABASE_URL=your_supabase_url
VITE_SUPABASE_ANON_KEY=your_supabase_key
```

🧠 Note

You are completely free to **clean up and remove any unnecessary environment variables** to simplify the setup and better suit the assignment use case.

Recommended

Remove all **OAuth-related variables** (Google configs)
Remove **Supabase client-side variables**
Remove **extra JWT configs like** `MAIN_SITE_JWT_SECRET`
Keep only:

- **DATABASE_URL**
- **Simple JWT auth (**`JWT_SECRET`**)**
- **Basic server config**

Simplify auth to **basic JWT login (no OAuth)**
Focus on **core product functionality instead of external integration**